CrumbTraits - A Route Tracking Application

Group Members:
Adam James Roy, Ayra Baig, Ibrahim Sajid

July 10, 2026

Abstract

CrumbTrails is an Android application that aims to solve one problem for runners, drivers, motorcyclists, and many more: tracking your aimless routes. Whether it is a joyride in a car or trying to find a more interesting running route, CrumbTrails will remember the trail that you have traveled, allowing users to look back on their route and place photos along that path automatically. Once having saved a route, these are also able to be shared amongst other users in the CrumbTrail community, finding more beautiful scenery, tasty pizza joints... Only your imagination is the limit!

Contents

1 Project Overview

1.1 Main Features and Functionality

1.2 Motivation for Creation

2 Technical Architecture

This Android application, built in Android Studio, utilizes a one activity architecture, local database where applicable, as well as cloud database base to focus on giving the user a lightning fast UI experience. Choices for external and internal APIs reflect these design decisions also allowing for scalability and maintainability in mind. These choices consist of Room, a local database option which offers an abstraction layer on top of SQlite3. Google's Firebase that has the role of managing authentication, as well as creating the environment for users to build a route-sharing community. The internal API was designed around a MVVM (model-view-view model) architecture to maintain a highly readable, scalable structure.

2.1 Frontend

Each feature that has been implemented that is directed interfaced with by the user is found in the **featuresAPI** directory. Each directory name corresponds with the name of the feature/function and consists of their MVVM components. This gives flexibilty to decouple the Jetpack Compose functions from the logic, as well as functions that will directly interface with the Room database. In the case of complex views such as the **history/ui**, this may be further decoupled to create smaller UI componenets.

2.2 Backend

Managing data locally was executed by creating a Room database, which mentioned above, uses SQLite3 as its functional base. Made of three distinct parts, Room is utilized to perform typical CRUD operation on the table containing the user's tracked routes, the time that the route was taken as well as directory paths of photos that were added post-trip. This project has one table for the routes, therefore, three main componenets; the **TrackedRouteEntity**, **TrackedRouteDao**, and **TrackedRouteRepository**. These are then connected via the **RouteTrackerDatabase**, which is then instantiated in the history tab. By using the functions supplied in the DAO, or data access object, the frontend as a fully decoupled layer that can access the database through the SQlite3 queries.

Important Data Classes

When saving each route to the local Room database, the data class below, **TrackedRoute**, was created:

```
data class TrackedRoute(
    val id: String,
    val tripName: String,
    val startedAtEpochMillis: Long,
    val endedAtEpochMillis: Long,
    val trackedRoute: List<LatLng>
)
```

The tracked data class has a couple unique traits in the form of its parameters. First, when the time stamp for beginning the trip and ending is measured by **Epoch time**. This is the time elapsed in milliseconds from January 1st, 1970. These two long values will be used to determine the time of the trip, and how long the trip lasted for. Secondly, the trackedRoute parameter is a list of tuples representing the gathered points latitude and longitude values (decimal form). Entering this into the database then requires encoding of the trackedRoute LatLng list, into a string using Google Maps PolyUtil API. *See Backend Implementation

3 API Design

```
java
├─ com.example.routetracker
│  ├─ ui.theme
│  │  ├─ Color.kt
│  │  ├─ Theme.kt
│  │  └─ Type.kt
│  ├─ utils
│  │  ├─ Converters
│  │  └─ PhotoDownsizer
│  └─ MainActivity.kt
├─ data
│  ├─ local
│  │  ├─ track
│  │  │  ├─ TrackedRoute
│  │  │  ├─ TrackedRouteDao
│  │  │  ├─ TrackedRouteEntity
│  │  │  └─ TrackedRouteRepository
│  │  └─ RouteTrackerDatabase
│  └─ location
│     └─ MapRepository
└─ featuresAPI
   ├─ authentication
   ├─ feed
   ├─ history
   ├─ navigation
   ├─ settings
   ├─ shared
   └─ tracking
```

Architecture Notes

Directory Overview:
This package structure separates core app logic from the modular feature APIs.

Key Components:

- `data/`: Handles local Room databases and repository logic.
- `featuresAPI/`: Contains isolated feature modules to support scalability. Each directory follows MVVM format.

NOTE: Other files not included here have been modified such as gradle build files. See Section 4 Requirements and Installation Instructions for more details.

3.1 Backend Implementaion

The following are functions that are substantial to for understanding the design choices taken for this project.

Function: `insert(route: TrackedRouteEntity)`

Uses the Room annotation **@Insert** to indicate that will be inserted into the database.
Source: data/local/track/TrackedRouteDao, line 13

Parameters

Name	Type	Description
`route`	TrackedRouteEntity	An instance of the corresponding entity

Returns `Unit` - The equivalent of void

Function: `getAll() : Flow<List<TrackedRouteEntity>>`

Uses the Room annotation **@Query** with the accompanying argument to retrieve the database elements in descending order.
Source: data/local/track/TrackedRouteDao, line 17

Parameters

Name	Type	Description
`No parameters`		

Returns `Flow<List<TrackedRouteEntity>>` - A flow list of the database's stored routes. Note that this is different from a normal list as this remains active to adjust to any database changes. Mainly used for displaying the user's past routes in the History page.

Function: `getById(id: String): TrackedRouteEntity?`

Retrieves the route data from a specific ID. This has been implemented mainly for sending the route do the Firebase server to share to the feed page.
Source: data/local/track/TrackedRouteDao, line 21

Parameters

Name	Type	Description
`id`	String	ID saved in the route entity

Returns `TrackedRouteEntity?` - The route entity is returned if the id exists in the database, Unit if not.

Function: `getById(id: String): TrackedRouteEntity?`

Retrieves the route data from a specific ID. This has been implemented mainly for sending the route do the Firebase server to share to the feed page.
Source: data/local/track/TrackedRouteDao, line 21

Parameters

Name	Type	Description
`id`	String	ID saved in the route entity

Returns `TrackedRouteEntity?` - The route entity is returned if the id exists in the database, Unit if not.

3.2 Frontend Implementation

Similar to the previous Backend Implementation section, the frontend

Function: `InsertEntry(item, code, cost, imagePath)`

A Function Description
Source: crudAPI/crudMenu/create.go, line 15

Parameters

Name	Type	Description
`item`	string	Menu item name
`code`	string	Unique item code
`cost`	cost	Cost of item in Kyat
`imagePath`	string	String for the path to image in Render

Returns

4 Requirements and Installation Instructions

4.1 System Requirements

Below are the requirements to build, run, and develop the project.

Development Environment

Requirement	Version / Detail
OS	Windows, macOS, or Linux (Android Studio compatible OS)
IDE	Android Studio (recent version, compatible with AGP 9.1.1 / Gradle 9.3.1)
JDK	**JDK 21**
Android Gradle Plugin (AGP)	9.1.1
Kotlin	2.2.10
KSP	2.2.10-2.0.2

Android SDK / Device Requirements

Setting	Value
`compileSdk`	37
`targetSdk`	36
`minSdk`	34 (Android 14+)
Java source/target compatibility	Java 11
Compose	Enabled (`buildFeatures.compose = true`)
BuildConfig	Enabled

Since `minSdk = 34`, the app only installs/runs on devices running **Android 14 (API 34) or newer**.

Required Gradle/Android Plugins

- `com.android.application` — 9.1.1
- `org.jetbrains.kotlin.plugin.compose` — 2.2.10
- `com.google.devtools.ksp` — 2.2.10-2.0.2 (annotation processing for Room)
- `com.google.android.libraries.mapsplatform.secrets-gradle-plugin` — 2.0.1 (used for taking the Google Maps API key from `local.properties`)
- `com.google.gms.google-services` — 4.4.2 (Firebase configuration)

External Services / Necessary Accounts

Two files are deliberately excluded from the repo (`.gitignore`) and **must be supplied locally** before the project will build/run:

1. `local.properties` (project root) — must define a Google Maps API key, e.g.:

```
GOOGLE_MAPS_API_KEY=your_key_here
```

Consumed by the secrets-gradle-plugin and injected into the manifest placeholder `${GOOGLE_MAPS_API_KEY}`.

2. `app/google-services.json` — Firebase project configuration file, downloaded from the Firebase console. Required for Firebase Auth, Realtime Database, and Storage to function.

You will therefore need:

- A **Google Cloud / Maps Platform** project with the Maps SDK for Android enabled (billing-enabled API key).
- A **Firebase** project with Authentication, Realtime Database, and Storage enabled.

App Permissions (Pre-declared in AndroidManifest.xml)

- `ACCESS_FINE_LOCATION`
- `ACCESS_COARSE_LOCATION`
- `INTERNET`
- `FOREGROUND_SERVICE`
- `FOREGROUND_SERVICE_LOCATION`
- `POST_NOTIFICATIONS`

(Used for GPS route tracking via a foreground `TrackingService`, and notifying the user while tracking is active.)

Library Dependencies

AndroidX / Jetpack

Library	Version
androidx.core:core-ktx	1.19.0
androidx.lifecycle:lifecycle-runtime-ktx	2.11.0
androidx.lifecycle:lifecycle-viewmodel-compose	2.8.2
androidx.activity:activity-compose	1.13.0
androidx.compose:compose-bom (BOM)	2024.09.00
androidx.compose.ui:ui	via BOM
androidx.compose.ui:ui-graphics	via BOM
androidx.compose.ui:ui-tooling / ui-tooling-preview	via BOM
androidx.compose.material3:material3	via BOM
androidx.compose.material:material-icons-extended	via BOM
androidx.navigation:navigation-compose	2.9.8
androidx.room:room-runtime, room-ktx, room-compiler (KSP)	2.8.4
androidx.exifinterface:exifinterface	1.4.1
androidx.photopicker:photopicker-compose	1.0.0-alpha01

Google / Firebase / Maps

Library	**Version**
com.google.android.gms:play-services-location	21.3.0
com.google.android.gms:play-services-maps	19.0.0
com.google.maps.android:maps-compose	6.0.0
com.google.maps.android:android-maps-utils	3.20.1
com.google.firebase:firebase-bom (BOM)	33.1.2
com.google.firebase:firebase-auth-ktx	via BOM
com.google.firebase:firebase-storage	via BOM
com.google.firebase:firebase-database	via BOM
firebase-database-ktx (catalog entry)	22.0.1

paragraphUtilities

Library	**Version**
io.coil-kt:coil-compose (image loading)	2.7.0
com.google.code.gson:gson	2.10.1

4.2 Installation Instructions

1. Install **JDK 21**.
2. Install **Android Studio**.
3. Clone the repo.
4. Create a **Google Maps API key** and add it to `local.properties` as `GOOGLE_MAPS_API_KEY`.
5. Create a **Firebase project**, download `google-services.json`, and place it in `/app`.
6. Ensure a target device/emulator runs **Android 14 (API 34) or newer**.
7. Sync Gradle and build — internet access is required for Gradle/Maven dependency downloads and for the app itself (Maps, Firebase) at runtime.

5 Testing

5.1 Summary

The following is our testing method for ensuring a complete, safe application environment:

1.

5.2 Known Limitations

6 Attributions and Citations

6.1 Cited Works

As the project virtually entirely used Google API platforms, when the Android Developers Documentation lacked depth, Gemini was resorted to to answer lingering questions. When possible, learning from the documentation was most crucial to learning the ins and outs of Android Studio, Kotlin, and Jetpack Compose. The following were used in this development process:

1. **States With Jetpack Compose**: `https://developer.android.com/develop/ui/compose/state?hl=ja`
2. **Dialog Composables**: `https://developer.android.com/develop/ui/compose/components/dialog?hl=ja`
3. **Type Converters for Room**: `https://developer.android.com/training/data-storage/room/referencing-data?hl=ja`
4. **General Room Explanation (DAO, Entities, Repositories)**: `https://developer.android.com/training/data-storage/room/defining-data?hl=ja`
5. **Firebase - Application Connection**: `https://firebase.google.com/docs/auth/android/start?hl=ja`
6. **Embedded Photo Picker**: `https://developer.android.com/training/data-storage/shared/photo-picker/embedded?hl=ja`
7. **NavController and NavHost Setup**: `https://developer.android.com/codelabs/basic-android-kotlin-compo hl=ja`
8. **StateFlow and LiveData**: `https://developer.android.com/kotlin/flow/stateflow-and-sharedflow?hl=ja`
9. **Switching to Foreground Service**: `https://developer.android.com/develop/background-work/services/fgs/restrictions-bg-start`

6.2 Contributions

Adam Created the local Room database structure, their accompanying files, and focused on creating the layout of the API. Implemented the Composables necessary to get the general functionality including for tracking, history, feed, and authentication.

Ayra

Ibrahim ☺